

密码学基石与AI安全

区块链原理与应用



主讲教师：黎方正

TEL:13907317103

本次课内容概览



理论基石：三大密码学原语

- ✓ 哈希函数 (Hash Functions)
- ✓ 非对称加密 (Asymmetric Encryption)
- ✓ 数字签名 (Digital Signatures)



前沿融合：隐私计算与AI

- ✓ 零知识证明 (ZKP)
- ✓ 同态加密 (Homomorphic Encryption)
- ✓ 应用场景：医疗、金融



动手实践：代码实现

- ✓ 哈希计算实战
- ✓ 数据加密与解密流程
- ✓ 数字签名生成与验证

从信任机器到可信计算



区块链本质：去中心化共识

在没有中心权威的网络环境中，通过算法建立互不信任节点间的共识机制，实现价值的点对点传输。



核心挑战：安全与确权

解决三大核心问题：确保数据不可篡改、身份真实可验证、交易操作获得用户授权。



密码学：信任的源代码

提供数学上可证明的安全工具，是构建区块链“信任机器”的底层逻辑与核心基石。



形象化理解

哈希函数 = 数据DNA/指纹；非对称加密 = 特制锁与钥匙；数字签名 = 可验证的亲手印章。

哈希函数：数据的“数字指纹”

核心定义

将任意长度的输入数据转换为固定长度字符串（哈希值）的单向函数，如同数据的“X光片”。

四大关键特性



确定性：相同输入永远产生相同输出。



雪崩效应：输入微小改动，输出截然不同。



单向性：无法从输出反推输入，保障隐私。



抗碰撞性：难以找到两个不同输入得到相同输出。



区块链核心应用场景



数据完整性校验

验证文件或区块内容是否被篡改，确保数据未被破坏。



构建默克尔树 (Merkle Tree)

高效、安全地汇总和验证大量交易，是轻量级节点同步的基础。



工作量证明 (PoW)

比特币挖矿的核心机制，通过寻找特定哈希值来确认记账权。

非对称加密：公钥与私钥的魔法

核心机制：公钥对体系

一对密钥的协作

公钥（Public Key）公开给所有人；私钥（Private Key）必须严格保密，永不泄露。

非对称加密流程

发送方使用接收方的公钥加密信息，密文生成后，只有对应的私钥才能解密，确保信息安全。

解决密钥交换难题

无需提前线下共享密钥，即可在不安全的网络环境中实现安全的端到端通信。

区块链场景应用

资产控制权凭证

公钥哈希生成钱包地址（收款），私钥作为数字签名的唯一依据，掌握资产的绝对控制权。

链上链下安全通信

在跨链消息传递或链下数据交互中，利用非对称加密技术对敏感业务数据进行加密保护。

安全警示：

私钥是资产的唯一拥有权证明，一旦丢失或泄露，资产将无法找回且不可逆转。务必通过助记词等方式进行多重备份。

数字签名：身份与承诺的证明

核心安全目标

- 真实性：确认发送者身份
- 完整性：确保信息未被篡改
- 不可否认性：承诺不可撤销

区块链场景应用



交易授权

每笔链上交易必须由私钥签名，确保资金所有权转移的合法性。



智能合约调用

对合约函数的每次调用都需签名授权，防止未授权的恶意操作。

01. 发送方签名

1. 计算原始信息的哈希值(H)
2. 使用私钥(Private Key)加密H
3. 生成数字签名并与原文发送



02. 接收方解密

1. 收到信息与数字签名
2. 使用公钥(Public Key)解密签名，得到哈希值A
3. 计算收到信息的哈希值B



03. 结果验证

1. 比对哈希值 A 与 B
2. 若一致：信息真实未篡改
3. 若不一致：信息已被破坏

新挑战：数据“可用”与“不可见”的矛盾



AI 训练刚需：海量数据

AI 模型的精准度依赖于高质量数据的投喂。医疗记录、财务信息等敏感数据往往蕴含巨大价值，但同时也涉及严格的隐私保护要求。



传统模式的局限性

直接共享数据存在严重的泄露风险；区块链虽保证透明可信，但链上数据的公开性使其难以直接承载敏感隐私信息。



隐私计算的核心目标

打破数据孤岛，实现数据的“可用不可见”。在加密或隐匿状态下完成计算，确保数据价值流动的同时保护隐私安全。



核心矛盾：如何在释放数据价值（可用性）的同时，确保数据不被非法窥探（不可见性）？

零知识证明：我证明给你看，但什么都不告诉你



核心思想：色盲朋友辨色实验

如何向色盲朋友证明两个球（红、绿）颜色不同，而不透露具体颜色？通过一系列验证步骤，让他确信“颜色不同”这一事实，却依然无法得知任何一个球的具体颜色。这就是“零知识”的含义：验证事实，隐藏细节。



场景一：模型可信推断 (AI+区块链)

AI服务方可以向用户证明模型预测是“用某个特定模型”执行的，而无需公开核心的模型权重。既保护了商业机密，又建立了用户信任。



场景二：隐私数据验证 (DeFi/信贷)

用户可以向信贷机构证明自己的收入“超过某个阈值”，而无需提供具体工资单。实现了数据的“可用不可见”，极大地保护了个人隐私。

同态加密：在密文上直接运算



核心数学原理

$$\text{Decrypt}(F(\text{Encrypt}(\text{data}))) = F(\text{data})$$



核心思想与分类

允许对加密数据进行计算，解密后的结果与处理原始数据的结果一致，实现“可用不可见”。



- 部分同态 (PHE)：仅支持加法或乘法中的一种
- 些许同态 (SHE)：支持有限次数的加法和乘法
- 全同态 (FHE)：支持任意复杂的函数运算



AI 融合关键场景



加密数据协同训练

多机构联合训练AI模型，全程数据加密，打破数据孤岛。



隐私智能合约

区块链合约直接处理加密输入并输出加密结果，保护商业机密。

隐私计算的价值落地

医疗健康领域

行业痛点：数据孤岛与协作难

病历数据分散在各医院，监管严格导致跨机构研究协作困难，AI模型训练缺乏足够数据。

解决方案：多方安全计算 + 区块链

构建跨医院联合AI训练平台，利用同态加密实现“数据可用不可见”；通过区块链记录授权与贡献度量，确保模型透明可信。

核心价值：加速科研与隐私保护

在严格保护患者隐私的前提下，显著加速疾病机理研究，提升AI辅助诊断的准确性。

金融服务领域

行业痛点：风控数据缺失与共享难

风控模型需要多维度数据，但出于合规与竞争原因，金融机构间数据不愿或不能直接共享。

解决方案：零知识证明技术

应用零知识证明技术，允许用户在不透露具体资产和负债细节的情况下，向银行证明其偿债能力符合贷款要求。

核心价值：精准风控与数据主权

提升风控模型的精准度，扩大金融服务覆盖范围，同时保障用户对自身财务数据的绝对主权。

动手时间：用Python实现密码学核心操作



实践目标

将抽象的密码学理论转化为具体代码，巩固知识并获得直接的编程体验。



环境配置

基础环境：Python 3.x
依赖库：hashlib, cryptography

核心实践任务列表



01. 哈希计算

实现文件完整性校验，并模拟默克尔树节点的生成过程。



02. 非对称加密

使用 RSA 算法生成公私钥对，完成数据的加密与解密操作。



03. 数字签名

实现数字签名的生成，并验证签名的有效性，确保数据来源可靠。

任务1：计算文件哈希与默克尔树节点

字符串哈希计算

```
# 1. 计算字符串哈希
data = "Blockchain2026"
hash_hex=
hashlib.sha256(data.encode()).hexdigest()
print(f"SHA-256: {hash_hex}")
```

文件哈希计算函数

```
# 2. 计算文件哈希
def get_file_hash(filename):
    with open(filename, 'rb') as f:
        return hashlib.sha256(f.read()).hexdigest()
```

模拟默克尔树节点

```
# 3. 模拟父节点生成
left = sha256(b"left").hexdigest()
right = sha256(b"right").hexdigest()
parent =
sha256((left+right).encode()).hexdigest()
```



思考：哈希值在区块链中如何用于快速验证大量数据的完整性？默克尔树结构是如何帮助区块链在海量交易中快速定位和验证被篡改的数据的？

任务1：计算文件哈希与默克尔树节点



字符串哈希计算

```
data = "Blockchain2026"  
hash_hex = hashlib.sha256(data.encode()).hexdigest()  
print(f"SHA-256: {hash_hex}")
```

作用：计算字符串"Blockchain2026"的SHA-256哈希值

详细解释：

`data.encode()`：将字符串转换为字节（因为哈希函数处理字节数据）

`hashlib.sha256()`：创建SHA-256哈希对象

`.hexdigest()`：返回十六进制格式的哈希值

输出示例：类似"a1b2c3d4e5f6..."的64个十六进制字符

区块链意义：这是最基本的数据指纹生成，区块链中每个交易、每个区块都有唯一的哈希

任务1：计算文件哈希与默克尔树节点



文件哈希计算函数

```
def get_file_hash(filename):  
    with open(filename, 'rb') as f:  
        return hashlib.sha256(f.read()).hexdigest()
```

作用：定义一个函数来计算整个文件的SHA-256哈希

详细解释：

`open(filename, 'rb')`: 以二进制只读模式打开文件

`f.read()`: 读取文件的全部内容

`hashlib.sha256(f.read()).hexdigest()`: 计算文件内容的哈希

区块链应用：

验证文件完整性（下载后比较哈希值）

数字版权保护：将文件哈希存储在区块链上作为证据

智能合约验证：通过哈希验证文件的真实性

任务1：计算文件哈希与默克尔树节点

模拟默克尔节点树

```
left_hash = hashlib.sha256(b"left_data").hexdigest()
right_hash = hashlib.sha256(b"right_data").hexdigest()
parent_hash = hashlib.sha256((left_hash + right_hash).encode()).hexdigest()
```

作用：模拟默克尔树（Merkle Tree）中父节点哈希的计算

详细解释：

计算左子节点哈希：hash("left_data")→ left_hash

计算右子节点哈希：hash("right_data")→ right_hash

计算父节点哈希：hash(left_hash + right_hash)→ parent_hash

区块链中的重要性：

默克尔树允许高效验证大量数据的完整性

比特币和以太坊都使用默克尔树来组织交易

轻节点只需验证根哈希就能确认所有交易的存在性

任务2：生成密钥对与加密解密

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import serialization, hashes

# 1. 生成密钥对 (2048位)
private_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
public_key = private_key.public_key()

# 2. 加密 (使用公钥)
message = b"A secret message for blockchain."
ciphertext = public_key.encrypt(message, padding.OAEP(...))

# 3. 解密 (使用私钥)
decrypted_msg = private_key.decrypt(ciphertext, padding.OAEP(...))
print(decrypted_msg.decode()) # 输出: A secret message...
```



注意：永远保管好你的私钥！私钥是你数字资产和身份的唯一凭证。

任务2：生成密钥对与加密解密

1. 生成密钥对 (2048位)

```
private_key= rsa.generate_private_key(public_exponent=65537, key_size=2048)
```

```
public_key= private_key.public_key()
```

`public_exponent=65537`: 常用的公钥指数（`e`值）。这个数字是质数，二进制中只有两个1，计算效率高

`key_size=2048`: 密钥长度为2048位。这是目前推荐的安全长度

`backend=default_backend()`: 指定加密库的后端实现

生成的`private_key`包含：私钥参数（`p, q, d`）和公钥参数（`n, e`）

`public_key = private_key.public_key()`: 从私钥提取公钥参数（`n, e`）

任务3：创建签名与验证签名



1. 使用私钥创建签名

```
hash_alg= hashes.SHA256()
signature= private_key.sign(
message, padding.PSS(...), hash_alg
)
```

2. 使用公钥验证签名

```
try:
public_key.verify(signature, message, padding.PSS(...), hash_alg)
print("验证成功: 身份与完整性确认")
except InvalidSignature:
print("验证失败: 消息被篡改或伪造")
```



创建签名 (Sign)

发送方使用私钥对消息摘要进行加密。这一过程生成了签名，证明了消息的来源身份。



验证签名 (Verify)

接收方使用公钥解密并验证签名。若通过，说明消息未被篡改且来源可信；否则抛出异常。

回顾与思考



信任基石

哈希、非对称加密、数字签名是构建区块链可信环境的三大支柱。



技术进化

零知识证明、同态加密等隐私计算技术，是区块链赋能AI数据协作的关键。



未来展望

密码学是“可信AI”和“数据要素化”时代的基础设施。



场景探索

除了医疗和金融，隐私计算还能在哪些“区块链+AI”的场景中发挥关键作用？



方案优化

同态加密目前计算开销很大，如何将其与区块链的可验证计算结合，设计更实用的方案？

问题与讨论



开放提问时间 (Q&A)

欢迎大家提出任何疑问，我们一起交流探讨。



下次课程预告

深入探讨区块链核心——“共识机制”，以及AI技术如何对其进行优化。



保持联系

课后交流请联系：13907317103@139.com。